TrustGraph: Distrust-Aware Trust-Weighted Introduction Routing for Startup Ecosystem Networking

Vikkram S L, Tarun Srikumar
Verllore Institute of Technology
Chennai, India
Email: vikkram.sl2023@vitstudent.ac.in, tarun.srikumar2023@vitstudent.ac.in

***Abstract*—Warm introductions are a core mechanism for startup fundraising and strategic partnerships, but existing networking systems do not jointly model directional trust quality, conflict signals, and intermediary workflow accountability. This paper presents TrustGraph, a microservice-based trust-intelligence platform that combines (i) trust-weighted multi-hop path discovery over a property graph, (ii) distrust-aware path filtering and reputation penalties, and (iii) a sequential approval workflow that enforces intermediary accountability before introduction completion. The implementation uses JanusGraph, Cassandra, Elasticsearch, MongoDB, Redis, and event-driven coordination through Kafka-compatible messaging. We evaluate the system using 25,200 synthetic trials across three conflict regimes and five ablation variants. Relative to shortest-path baseline (V0), the full model (V4) improves acceptance rate from 14.56% to 22.74% and completion rate from 11.07% to 17.10% (both FDR-adjusted $q < 10^{-17}$), while eliminating blocked-path exposure in created intros (48.31% to 0%). The gain introduces latency overhead (mean discovery latency 26.60 ms to 36.11 ms), quantifying a safety-performance trade-off that is favorable for intro quality.**

***Index Terms*—trust graph, path discovery, warm introductions, reputation systems, NoSQL, microservices, statistical evaluation**

I. Introduction

Startup relationship formation is strongly influenced by trusted intermediaries rather than cold outreach. In early-stage ecosystems, introductions carry reputational risk for both the introducer and the recipient. Existing networking platforms optimize for visibility, response volume, or graph reach, but they usually provide limited support for directional trust quality, explicit negative ties, and accountable execution of multi-party introductions. As a result, systems can surface paths that are topologically short but socially brittle.

TrustGraph addresses this gap through a distrust-aware trust graph coupled with a workflow engine that enforces intermediary accountability. The platform is implemented as a polyglot NoSQL microservice architecture with separable responsibilities for graph reasoning, workflow orchestration, entity search, and asynchronous event propagation. This design enables low-latency recommendation while preserving auditable state transitions for introduction requests.

A. Motivating Failure Modes

Three practical failure modes motivate the design. First, trust-only ranking can over-select connectors who are highly central but currently involved in unresolved conflicts, increasing downstream decline risk. Second, path-selection logic and execution logic are often decoupled, so an apparently strong chain may fail because intermediary approvals are not sequenced or attributable. Third, systems without explicit distrust modeling can optimize conversion metrics at the expense of long-term network reliability by repeatedly routing through contentious actors.

B. Design Requirements

The system is built around five requirements: (R1) directional trust estimation from observable interaction signals, (R2) explicit handling of negative ties and dispute state, (R3) bounded-hop discovery suitable for interactive applications, (R4) workflow-constrained completion with strict expected-approver transitions, and (R5) reproducible evaluation under controlled conflict regimes.

The main contributions are as follows:

1) A distrust-aware trust-weighted path discovery model that ranks introduction chains by trust quality, interaction recency, relationship strength, and mutual-trust structure, while applying hard governance filters.
2) A workflow-constrained completion mechanism that enforces sequential intermediary approvals and emits auditable transition events before graph-side completion updates.
3) A production-aligned microservice implementation over JanusGraph, Cassandra, MongoDB, Elasticsearch, Redis, and Kafka-compatible messaging, including idempotent event processing boundaries.
4) A reproducible simulation and ablation protocol with 25,200 trials across controlled conflict regimes, plus pre-simulation quality audits for topology realism and governance pressure.
5) A statistical testing pipeline with multiplicity control and effect-size reporting to quantify both quality gains and latency overhead.

II. Related Work

Graph-based social recommendation and trust propagation have been studied across social networks, e-commerce, and peer-to-peer systems [1]–[3]. Reputation computation and

graph centrality are commonly used to estimate influence and reliability [4], [5]. Modern production systems combine graph storage, event-driven microservices, and search indexing [6], [7]. However, fewer systems explicitly incorporate distrust filtering and enforce approval workflows for warm-introduction integrity.

A. Trust and Reputation in Networked Systems

Prior trust systems emphasize propagation, local transitivity assumptions, or global spectral ranking. These approaches are effective for robustness against spam and low-quality edges, but they often focus on scalar trust intensity and may not preserve direction-specific context required for high-stakes introductions. In startup networking, trust from u to v is frequently asymmetric because it depends on sector fit, prior collaboration outcomes, and perceived reputational exposure.

B. Negative Ties and Conflict Signals

Negative relations are often modeled as weak penalties, edge sign inversions, or exclusion constraints at training time. In operational introduction routing, unresolved conflict can be better treated as a governance boundary rather than a soft preference. This motivates TrustGraph's distinction between active undisputed distrust (hard exclusion) and disputed or inactive distrust (non-blocking signal), reducing false positives caused by stale or contested conflict edges.

C. Workflow-Aware Social Coordination

Many social-routing pipelines stop at recommendation quality and do not model how introductions are executed through human approvals. Workflow engines in enterprise systems provide sequencing and auditability, but are rarely linked directly to graph-state evolution for trust propagation. TrustGraph integrates both layers: routing quality determines candidate selection, while workflow correctness determines whether an intro can complete and trigger graph updates.

D. Gap Summary

Existing work offers strong foundations for trust scoring, graph ranking, and distributed systems design, yet practical warm-introduction integrity requires joint treatment of these dimensions. TrustGraph contributes this integration by coupling distrust governance, trust-aware ranking, and workflow accountability in one deployable architecture.

III. Problem Formulation

Let users be modeled as a directed property graph $G = (V, E_T, E_D, E_V)$, where E_T are accepted trust edges, E_D are distrust edges, and E_V are optional vouch edges.

Given a founder node $v_f \in V$ and investor node $v_i \in V$, the objective is to identify an ordered path $P = (v_0, v_1, ..., v_h)$ such that:

$$v_0 = v_f, \quad v_h = v_i, \quad h \leq H_{max} \tag{1}$$

and maximize path quality while satisfying governance constraints (no active undisputed distrust conflicts in path nodes).

Define the bounded candidate set:

$$\mathcal{P}_{fi}^{(H)} = \{P = (v_0, \ldots, v_h) \mid v_0 = v_f,\ v_h = v_i,\ h \leq H_{max},\ (v_{k-1}, v_k) \in \tag{2}$$

For each node v, let d_v^{act} denote the count of incoming active undisputed distrust edges. Path feasibility under governance is:

$$\gamma(P) = \prod_{v \in P} \mathbb{I}(d_v^{act} = 0) \tag{3}$$

where $\gamma(P) = 1$ indicates governance-valid paths.

The routing objective is:

$$P^* = \arg \max_{P \in \mathcal{P}_{fi}^{(H)}} [\gamma(P) \cdot (\text{score}(P) - \lambda_\rho \bar{\rho}(P))] \tag{4}$$

where $\bar{\rho}(P)$ is path-level reputation risk from distrust incidence and $\lambda_\rho \geq 0$ controls penalty strength.

Workflow execution is modeled as an ordered approver sequence $A = (a_1, \ldots, a_m)$. Let state $s_t \in \{\text{created}, \text{in_progress}, \text{declined}, \text{completed}\}$. A transition from step k to $k+1$ is valid only if actor a_k approves at time t and request version is current. This enforces a safety condition: no completion without full intermediary accountability.

IV. System Architecture

A. Service Decomposition

TrustGraph is implemented with six main services: API Gateway, Graph Service, Workflow Service, Search Service, Messaging Service, and Post Service. The API Gateway performs auth verification, request shaping, and coarse-grained throttling. Domain services remain independently deployable and communicate primarily through event contracts and minimal synchronous APIs.

Data ownership is split across JanusGraph (trust graph), MongoDB (workflow and content state), Elasticsearch (search projections), Redis (caching/rate limits), and Kafka-compatible event streaming. This polyglot decomposition separates traversal-heavy operations from document workflows and full-text retrieval.

TABLE I
PERSISTENT STORE MAPPING AND CONSISTENCY STRATEGY

Store	Primary records	Consistency mode
JanusGraph + Cassandra	Users, TRUSTS, DISTRUSTS, VOUCHES, edge features	Strong per write; eventual cross-service
MongoDB	Intro requests, approver logs, workflow snapshots	Document-level atomic
Elasticsearch	User/profile search documents, denormalized tags	Async projection
Redis	Hot-path query cache, counters, rate-limit buckets	TTL-based ephemeral
Kafka-compatible broker	Domain events, retry/dead-letter topics	At-least-once delivery

B. End-to-End Request Lifecycle

An introduction request follows six stages: (1) requester/target resolution via Search Service, (2) candidate path generation in Graph Service, (3) governance validation against active distrust state, (4) ranked candidate return, (5) sequential approval handling in Workflow Service, and (6) completion-triggered graph updates and event emission. Instrumentation spans each stage to isolate latency contributors and failure points.

C. Graph Service Responsibilities

The Graph Service manages vertices and edges, computes trust/reputation features, performs path discovery with bounded hop constraints, and executes scheduled trust maintenance jobs. It exposes APIs for trust edge updates, reputation refresh, and route recommendation. To reduce high-degree expansion cost, the service applies pre-filtering by minimum trust thresholds and edge staleness bounds before full ranking.

D. Workflow Service Responsibilities

The Workflow Service persists introduction requests, enforces expected-approver transitions, supports accept/decline actions, and triggers completion side effects (including direct trust edge enforcement between requester and investor). A versioned request document prevents stale writes. Duplicate action submissions are handled by idempotency keys keyed on request title, actor, decision, and version.

E. Search and Event Integration

The Search Service indexes entities and optionally reranks by graph proximity. Cross-service consistency is handled through asynchronous events and idempotent processing patterns. Projection lag is tolerated by design because path discovery is anchored to graph truth while search primarily accelerates candidate discovery.

F. Resilience, Observability, and Access Control

Operational resilience is achieved through retry with exponential backoff, dead-letter routing for non-transient failures, and bounded replay windows to avoid out-of-order amplification. Structured logs include request IDs, path IDs, and workflow correlation IDs. Metrics include discovery p95/p99 latency, workflow step failure rates, cache hit ratio, and consumer lag. Access control follows service-to-service JWT validation, scoped claims, and ownership checks for workflow actions.

V. Methodology

A. Edge Feature Construction

For each directed trust edge (u, v), the system maintains interaction count n_{uv}, successful introduction count c_{uv}^{succ}, total introduction attempts c_{uv}^{req}, and recency gap Δt_{uv} in days from last high-signal interaction. Features are normalized to $[0, 1]$:

$$f_{int}(u, v) = \min\left(1, \frac{\log(1 + n_{uv})}{\log(1 + n_{ref})}\right) \tag{5}$$

$$f_{succ}(u, v) = \frac{c_{uv}^{succ} + 1}{c_{uv}^{req} + 2} \tag{6}$$

$$f_{rec}(u, v) = \exp\left(-\frac{\Delta t_{uv}}{\tau}\right), \quad \tau = 90 \tag{7}$$

where n_{ref} is a saturation reference selected from historical percentile statistics.

B. Trust Score Model

For an accepted trust edge, the trust score is computed from interaction history, intro outcomes, and recency:

$$\begin{aligned}\text{trust}(u, v) = 0.40 \cdot f_{int}(u, v) + 0.40 \cdot f_{succ}(u, v) \\ + 0.20 \cdot f_{rec}(u, v)\end{aligned} \tag{8}$$

where f_{int} is interaction-derived contribution, f_{succ} is successful-introduction contribution, and f_{rec} is recency contribution in a 90-day decay window.

Relationship strength and recency edge factors are computed similarly and clipped to $[0, 1]$. Mutual trust factor is modeled as:

$$m_e = 1 + 0.5 \cdot \mathbb{I}\left((v, u) \in E_T\right) \tag{9}$$

so reciprocal ties receive a bounded boost.

C. Path Scoring

For a path P with h hops, path score is:

$$\text{score}(P) = \left[\frac{1}{h}\sum_{e \in P}\left(t_e \cdot s_e \cdot r_e \cdot m_e\right)\right] \cdot \pi(h) - \lambda_\rho \bar{\rho}(P) \tag{10}$$

where t_e is trust score, s_e is strength score, r_e is recency score, $m_e \in \{1.0, 1.5\}$ is mutual-trust boost, and $\pi(h)$ is hop penalty.

Hop penalty is defined as:

$$\pi(h) = \exp\left(-\alpha(h - 1)\right), \quad \alpha > 0 \tag{11}$$

which reduces over-long chains that are less likely to complete in practice.

Path-level reputation risk is the mean node penalty:

$$\bar{\rho}(P) = \frac{1}{|P|}\sum_{v \in P}\rho(v), \quad \rho(v) = \min(1, \beta\, d_v^{act}) \tag{12}$$

The risk term is only evaluated after hard governance filtering and acts as a tie-breaker among valid paths.

D. Distrust Governance

A candidate path is rejected if any path node has active undisputed distrust relationships. Disputed or inactive distrust edges are excluded from rejection logic. Additional reputation penalty is applied based on incoming active distrust count. This two-tier design separates strict safety constraints from softer prioritization signals.

E. Workflow-Constrained Completion

A request remains valid only if current action is performed by the expected intermediary index. Completion requires all intermediaries to approve in order; any decline terminates the request.

F. Candidate Enumeration and Ranking Procedure

Algorithm 1: Distrust-Aware Ranked Path Selection

Require: Founder v_f, investor v_i, max hops H_{max}, top-K
Ensure: Ordered candidate list $\mathcal{R}$

1: Enumerate simple trust-only paths $\mathcal{P}_{fi}^{(H)}$ with $h \leq H_{max}$
2: Remove paths with any active undisputed distrust node
3: **for** each remaining path P **do**
4: Compute edge factors (t_e, s_e, r_e, m_e) and hop penalty $\pi(h)$
5: Compute score(P) including risk tie-break penalty
6: **end for**
7: Sort by descending score, then ascending hop count, then lexical path ID
8: Return top-K ranked paths

G. Computational Considerations

Worst-case bounded path enumeration has exponential growth in hop bound, approximately $\mathcal{O}(b^{H_{max}})$ for branching factor b. In practice, TrustGraph constrains runtime through maximum hop limits, per-hop candidate pruning by minimum trust, cache-backed neighborhood expansion, and early governance rejection. These controls preserve interactive latency while retaining enough candidate diversity for robust ranking.

VI. Experimental Design and Simulation Plan

A. Research Questions

- RQ1: Does distrust-aware filtering improve intro completion quality compared to trust-only ranking?
- RQ2: Does mutual-trust and recency weighting improve path acceptance over shortest-path baselines?
- RQ3: What latency overhead is introduced by distrust-aware filtering and workflow governance?

RQ1 targets outcome quality under conflict pressure, RQ2 isolates ranking-feature contribution in the absence of hard governance, and RQ3 quantifies operational cost for deployment decisions.

B. Hypotheses

- H1: Full model yields higher intro completion rate than baseline shortest-path.
- H2: Full model yields higher acceptance rate than trust-only ranking.
- H3: Latency increase from distrust filtering remains within acceptable p95 bounds.

C. Ablation Variants

- V0: Shortest-path baseline (no trust weighting, no distrust filtering).
- V1: Trust-only ranking.
- V2: Trust + strength + recency + mutual boost.
- V3: V2 + distrust filtering.
- V4: Full model (V3 + workflow-constrained completion updates).

The variant progression is cumulative by design so each step adds one interpretable mechanism. This supports causal-style attribution for quality and latency differences.

D. Simulation Protocol

1) Seed graph and relationship state with controlled random seeds.
2) Materialize TRUSTS, DISTRUSTS, and reciprocal structures by condition.
3) Draw fixed founder-investor query batches per seed.
4) Execute all ablation variants on identical graph snapshots.
5) Select top-ranked route per variant and run workflow simulation.
6) Emit stage-level latency telemetry and workflow transition logs.
7) Aggregate per-trial outcomes into analysis-ready tables.
8) Repeat across independent seeds and conflict regimes.

Trials are run in a paired manner per query: each variant sees the same source-target pair and same underlying graph realization. This minimizes variance inflation from graph heterogeneity when comparing variants.

E. Synthetic Dataset Construction

The simulation dataset is generated by a TrustGraph-aligned pipeline model. Each generated graph contains 30 founders, 20 investors, and 60 connectors. Directed TRUSTS edges are sampled with condition-specific trust, strength, and recency distributions; reciprocal TRUSTS edges emerge probabilistically to model mutual trust. DISTRUSTS edges are sampled with condition-specific active/disputed rates to create low, moderate, and high conflict regimes.

We use 12 random seeds per condition and 140 founder-investor queries per seed. For each query, all five variants (V0-V4) are executed over identical graph state, producing:

$$3 \text{ conditions} \times 12 \text{ seeds} \times 140 \text{ queries} \times 5 \text{ variants} = 25{,}200 \text{ trials} \tag{13}$$

Each variant contributes 5,040 trial rows.

Each trial row stores condition ID, seed, source-target pair, selected path metadata, governance validity flags, workflow transition trace, and latency breakdown. This schema supports both endpoint-level analysis and stage-level diagnostic analysis.

F. Conflict Regime Parameterization

Conflict regimes are controlled through distrust density and active/undisputed ratios while preserving high baseline graph connectivity.

G. Workflow Outcome Simulator

For each selected path, intermediary approvals are simulated sequentially. Approval probability at hop position k is modeled with a calibrated logistic form:

$$p_{approve}^{(k)} = \sigma\left(\eta_0 + \eta_1 \,\text{score}(P) - \eta_2\, h - \eta_3\, \bar{\rho}(P) + \eta_4\, \mathbb{I}_{mutual}\right) \tag{14}$$

TABLE II
CONFLICT REGIME CONTROLS (TARGETED RANGES)

Parameter	C1 (Low)	C2 (Moderate)	C3 (High)
Mean trust target	0.72	0.64	0.57
Reciprocity target	0.57	0.52	0.46
Blocked-node ratio target	0.07	0.21	0.34
Active distrust density	Low	Medium	High
Disputed distrust share	Higher	Medium	Lower

where $\sigma(\cdot)$ is the sigmoid function and parameters (η_i) are fixed across variants. This design preserves a consistent behavioral model while allowing route quality to drive acceptance and completion differences.

H. Randomization and Control Strategy

Random seeds are fixed and versioned for full replay. Query order is shuffled per seed to reduce temporal ordering artifacts in cache-sensitive paths. All variants run against the same generated graph state before any variant-specific workflow side effects are applied, preventing contamination across ablations.

I. Pre-Simulation Data Quality Audit

Before running ablation trials, we executed a structural and statistical quality audit over generated graphs (12 seeds per condition, 160 sampled founder-investor pairs per seed). The audit checks topology realism, feature distributions, and conflict-governance separation.

Condition means follow the intended difficulty progression:

- Trust mean decreases from C1 to C3 (0.7186, 0.6422, 0.5727), with stable spread ($\sigma \approx 0.11$).
- Reciprocity decreases with conflict (0.5748, 0.5175, 0.4622).
- Blocked-node ratio increases by condition (0.0727, 0.2106, 0.3447).
- Distrust-safe path availability decreases (0.9167, 0.7474, 0.5219), while raw path availability remains high, indicating filtering pressure from governance rather than graph disconnection.
- A weighted non-governed selector (V2-style) chooses blocked top paths at rates 0.2620, 0.5839, and 0.8359 across C1-C3.

These checks indicate that synthetic graphs are neither trivially easy nor degenerate, and that pre-simulation conditions represent realistic gradients in trust quality and conflict pressure.

Additional checks confirmed that degree distributions remained heavy-tailed but bounded, average shortest-path distance stayed within practical warm-intro range, and candidate-path multiplicity was sufficient to test ranking behavior beyond shortest-hop tie cases.

J. Pipeline Processing Simulation

To mirror runtime behavior of the deployed architecture, each trial records stage-level processing latencies: graph traversal, distrust filtering, path scoring, workflow validation, and trust update propagation. Table III summarizes mean stage costs.

TABLE III
MEAN PROCESSING-STAGE LATENCY (MS)

Variant	Graph	Distrust	Scoring	Workflow	Trust Upd.	Total
V0	24.09	0.00	2.51	1.29	0.18	28.07
V2	24.06	0.00	5.60	1.68	0.20	31.54
V4	24.09	5.82	6.21	1.76	0.31	38.17

Stage instrumentation distinguishes discovery-time costs from post-selection workflow costs. This separation is important because operational SLAs may target discovery responsiveness while tolerating longer asynchronous completion processing.

K. Execution Environment and Artifacts

Simulation and analysis pipelines are implemented as versioned scripts with deterministic random seeds. Trial outputs, summary tables, and publication artifacts are generated from a single source-of-truth CSV layer to reduce hand-transcription errors. The pipeline emits both machine-readable outputs (CSV/JSON) and paper-ready tables/figures (LaTeX/Markdown).

VII. EVALUATION METRICS

A. Primary Metrics

Primary metrics quantify end-to-end introduction quality and runtime cost:

- Intro acceptance rate.
- Intro completion rate.
- Median time-to-intro completion.
- Mean/median path score of accepted paths.
- Endpoint latency (p50, p95, p99).

For variant v with N_v trials, acceptance and completion are computed as:

$$\text{AcceptRate}_v = \frac{\#\text{accepted}_v}{N_v}, \qquad \text{CompleteRate}_v = \frac{\#\text{completed}_v}{N_v} \tag{15}$$

We also report conditional completion among accepted intros:

$$\text{CondComplete}_v = \frac{\#\text{completed}_v}{\#\text{accepted}_v} \tag{16}$$

which separates path quality from workflow follow-through.

B. Safety and Governance Metrics

Safety-specific metrics track how effectively distrust governance avoids risky introductions:

- Blocked-path exposure among created intros.
- Path-found rate after governance filtering.
- Distrust-safe path availability per query.

Blocked-path exposure is defined as:

$$\text{BlockedExposure}_v = \frac{\#\text{created intros with blocked nodes}}{\#\text{created intros}} \tag{17}$$

This metric directly reflects whether generated routes violate governance intent.

C. Secondary Metrics

Secondary diagnostics capture longitudinal trust quality and conflict pressure:

- Distrust dispute ratio.
- Vouch-to-distrust ratio.
- Reputation stability index.

These measurements are used to verify that gains in acceptance and completion are not obtained by destabilizing trust dynamics.

VIII. Statistical Analysis Plan

A. Descriptive Reporting

For each metric and variant, report sample size n, mean, standard deviation, median, interquartile range, and 95% confidence interval where applicable. Binary outcomes are reported with Wilson confidence intervals; continuous outcomes use bootstrap confidence intervals when normality assumptions are weak.

B. Hypothesis Testing

- Binary outcomes (acceptance/completion): chi-square or Fisher exact test.
- Continuous outcomes (latency, path score): Welch t-test if normality is satisfied; Mann-Whitney U otherwise.
- Multi-variant comparisons: one-way ANOVA or Kruskal-Wallis; followed by post-hoc pairwise tests.

Normality diagnostics are assessed with Shapiro-Wilk tests and Q-Q inspection on sampled batches; variance equality is checked using Levene tests where relevant. Because latency distributions are right-skewed in practice, non-parametric tests are treated as default unless diagnostics strongly support parametric assumptions.

C. Effect Size and Multiplicity

Report effect sizes (Cohen's d, Cliff's delta, risk difference, odds ratio) and adjust multiple comparisons using Benjamini-Hochberg false discovery rate control.

D. Analysis Decision Rules

Two-sided tests are used throughout with nominal threshold $\alpha = 0.05$ before FDR adjustment. Primary claims require both statistical significance (FDR-adjusted $q < 0.05$) and practical relevance based on predeclared minimum effect criteria (for example, completion risk difference ≥ 0.03 absolute).

E. Robustness and Sensitivity Checks

Robustness analyses include (i) seed-level blocked bootstrap resampling, (ii) condition-stratified comparisons (C1-C3), and (iii) leave-one-seed-out sensitivity to identify seed-driven artifacts. These checks help confirm that observed gains are not concentrated in a narrow subset of graph realizations.

TABLE IV
Primary Outcomes Across Ablation Variants

Variant	n	Accept. Rate	Completion Rate	Median Time (h)	p95 Latency
V0	5040	14.56%	11.07%	9.01	34.86
V1	5040	15.10%	10.81%	9.44	36.15
V2	5040	17.44%	11.27%	10.08	37.81
V3	5040	22.44%	14.23%	10.59	46.48
V4	5040	22.74%	17.10%	9.66	46.80

TABLE V
Hypothesis Test Summary

Metric	Comparison	Test	Statistic	p-value	FDR-
Acceptance rate	V4 vs V1	χ^2	95.86	1.23×10^{-22}	2.96×10
Completion rate	V4 vs V0	χ^2	75.75	3.21×10^{-18}	5.79×10
Completion rate	V4 vs V2	χ^2	70.44	4.75×10^{-17}	9.00×10
Discovery latency	V4 vs V2	MW-U	20,763,133	$< 10^{-300}$	$< 10^{-3}$

IX. Results

A. Primary Outcomes

B. Statistical Test Summary

All listed comparisons remain significant after FDR adjustment, indicating that the observed gains are not artifacts of multiple testing. Effect sizes are practically meaningful for both acceptance/completion outcomes and latency shift.

C. Relative and Absolute Gains

Three patterns are consistent across outcomes:

- **Quality gains**: Compared to V0, V4 increases acceptance by $+8.18$ percentage points (14.56% to 22.74%) and completion by $+6.03$ points (11.07% to 17.10%).
- **Governance impact**: Compared to trust-only V1, V4 improves acceptance by $+7.64$ points and completion by $+6.29$ points.
- **Workflow benefit**: Relative to V2, V4 increases conditional completion among accepted intros from 64.62% to 75.22%.

These shifts are consistent with the core thesis: ranking quality alone improves top-of-funnel acceptance, while governance and workflow correctness are required to improve end-to-end completion.

D. Safety-Latency Frontier

Distrust governance introduces a measurable retrieval cost but substantially improves safety. Path-found rate decreases from 100% (V0-V2) to 72.06% (V3-V4), indicating that a non-trivial portion of candidate routes are filtered under conflict constraints. However, blocked-path exposure in created intros is eliminated (48.31% to 0%), showing that the filter enforces the intended policy in deployed decisions rather than only in offline scoring.

Latency overhead is concentrated in distrust checks and richer scoring stages (Table III), while graph traversal cost remains essentially stable. This indicates that performance optimization should target governance-index lookups and score-factor aggregation rather than traversal kernels.

E. Interpretation by Ablation Step

The staged ablation design enables mechanism attribution:

- V0→V1: trust-only ranking gives small acceptance gains with minimal completion lift.
- V1→V2: richer edge factors (strength, recency, mutuality) increase acceptance further, but completion remains constrained by downstream failures.
- V2→V3: hard distrust filtering drives the largest safety improvement and substantial acceptance uplift under conflict.
- V3→V4: workflow-constrained completion yields the largest incremental completion gain by reducing invalid transition paths.

This decomposition supports a practical architecture strategy: deploy distrust governance and workflow enforcement as first-class components rather than optional post-processing.

F. Figures (Placeholders)

- Fig. 1: Architecture and data flow diagram.
- Fig. 2: Acceptance and completion rate by variant with 95% CI.
- Fig. 3: Latency distribution by variant.
- Fig. 4: Path score distribution and distrust-filter impact.

X. Discussion

The ablation results show that distrust governance and workflow constraints are the dominant contributors to end-to-end intro quality. Pure trust weighting (V1/V2) improves acceptance relative to shortest-path routing but does not improve completion at the same magnitude as V4. The full model increases completion over trust-only V1 by an absolute 6.29 percentage points and over weighted V2 by 5.83 points, with strong statistical support.

Compared with trust-only and shortest-path baselines commonly used in related graph recommendation settings [2], [3], TrustGraph's processing is distinct in two ways:

- **Negative-signal integration**: active undisputed distrust edges are used as hard governance filters rather than only as weak penalties.
- **Workflow-coupled graph updates**: sequential intermediary validation is explicitly modeled and affects completion, trust propagation, and reliability outcomes.

A. Why Governance Improves Completion

The strongest completion gains appear only when distrust filtering and workflow constraints are jointly active. This suggests that many failures in trust-only pipelines are not due to poor path score estimation alone, but due to latent conflict risk and unstructured coordination. By excluding blocked intermediaries early, V3 and V4 reduce the probability of late-stage declines. By enforcing expected-approver ordering, V4 further reduces invalid transitions and stale-action failures.

B. Operational Implications

The efficiency trade-off is explicit. V4 adds approximately 9.51 ms mean discovery latency over V0 and 6.45 ms over V2, primarily from distrust checks and richer scoring. However, for completed intros, median completion time improves versus V2 (9.66 h vs 10.08 h), suggesting that safer path selection reduces downstream coordination failures. In product terms, a modest online latency increase can be justified when it reduces expensive human coordination churn.

C. Scalability and Cost Considerations

From the stage-level profile, graph traversal remains stable across variants while governance and scoring stages absorb most additional cost. This points to clear optimization targets: precomputed distrust aggregates, cache-aware risk lookups, and vectorized score computation over candidate batches. Because the largest incremental gains occur between V2 and V4, deployments with strict latency budgets can still adopt partial governance (V3-like) before enabling full workflow coupling.

D. Product and Policy Design Implications

TrustGraph supports an explicit policy stance: introduction quality should prioritize reliability and reputational safety over raw path availability. This policy is observable through zero blocked-path exposure in generated intros under V4. For platform operators, this translates into clearer user expectations, auditable moderation decisions, and improved trust in recommendation outputs.

XI. Threats to Validity

A. Internal Validity

Simulation outcomes depend on generative assumptions for trust, conflict, and approval behavior. If these assumptions are misspecified, estimated gains may be biased. We mitigate this risk with pre-simulation audits, paired-variant execution on identical graph snapshots, and seed-stratified robustness checks.

B. External Validity

Synthetic graph topology may not fully represent all startup ecosystems, especially niche sectors with sparse ties or highly clustered investor communities. While condition regimes model broad conflict gradients, real platforms may exhibit different reciprocity and distrust dynamics. Field validation is therefore required before production policy hardening.

C. Construct Validity

Acceptance and completion are operational proxies for introduction quality, but they do not capture all business outcomes (for example, funding conversion quality, long-term partnership value, or user satisfaction). Future studies should include post-intro value metrics and retention outcomes to strengthen construct alignment.

D. Reliability and Reproducibility

Although seeded simulations are deterministic, infrastructure-level effects such as asynchronous queue lag can influence measured latency distributions. We reduce this risk with repeated seeds, stage-level instrumentation, and artifact generation from a single canonical trial table. Reproducibility scripts should be version-locked with explicit dependency manifests.

XII. Reproducibility and Artifact Availability

The research package includes simulation drivers, statistical analysis templates, and generated outputs for tables and summaries. Reproducibility relies on fixed seeds, explicit variant flags, and deterministic post-processing scripts. To facilitate peer verification, each trial row contains condition, seed, query ID, variant, selected-path metadata, workflow outcomes, and latency traces.

Recommended artifact bundle for submission includes: (i) source code for data generation and evaluation, (ii) immutable snapshot of trial CSV files, (iii) scripts that regenerate all manuscript tables, and (iv) a short execution checklist with expected checksums for key outputs.

XIII. Conclusion and Future Work

TrustGraph combines distrust-aware trust modeling with workflow accountability in a scalable microservice architecture. In a 25,200-trial simulation study, the full model significantly improves acceptance and completion outcomes over shortest-path, trust-only, and weighted-without-governance baselines, while introducing a quantifiable latency overhead.

The results support a practical design conclusion: integrating explicit conflict governance into graph routing is beneficial for intro reliability even when it reduces path availability. The ablation sequence also clarifies that routing and workflow should be evaluated jointly, because improvements in candidate quality do not automatically translate to completed introductions.

Future work includes calibrated learned ranking, temporal graph embeddings, adaptive distrust resolution policies, online A/B validation with real platform traffic, and fairness-aware auditing of conflict filters across user cohorts.

Acknowledgment

This section is optional. Add sponsors, mentors, and infrastructure support as needed.

References

[1] M. Newman, *Networks*, 2nd ed. Oxford University Press, 2018.
[2] S. D. Kamvar, M. T. Schlosser, and H. Garcia-Molina, "The eigentrust algorithm for reputation management in p2p networks," in *Proceedings of the 12th International Conference on World Wide Web*, 2003, pp. 640–651.
[3] J. Golbeck, "Generating predictive movie recommendations from trust in social networks," *Lecture Notes in Computer Science*, vol. 3980, pp. 93–104, 2006.
[4] L. C. Freeman, "Centrality in social networks: Conceptual clarification," *Social Networks*, vol. 1, no. 3, pp. 215–239, 1978.
[5] L. Page, S. Brin, R. Motwani, and T. Winograd, "The pagerank citation ranking: Bringing order to the web," *Technical Report*, 1999, stanford InfoLab.
[6] S. Newman, *Building Microservices*. O'Reilly Media, 2015.
[7] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.